

Projet : Développement du jeu Snake en 8086

1. Objectif

L'objectif de ce projet est de réaliser un jeu Snake en langage assembleur 8086 en utilisant un environnement DOS.

L'étudiant devra :

- Comprendre le fonctionnement du microprocesseur 8086
- Manipuler les interruptions BIOS et DOS
- Implémenter un affichage graphique
- Gérer les entrées clavier
- Créer une boucle de jeu

2. Guide d'installation

1. Installer DOSBox-X
2. Installer NASM
3. Créer un dossier de travail, par exemple :

C:\dos\snake

4. Monter le dossier dans DOSBox :

mount c C:\dos\snake
c:

3. Code complet du jeu Snake

Créer un fichier nommé `snake.asm` et copier le code suivant :

```
org 100h
jmp start

; --- DONNÉES ---
snake_x db 40, 39, 38, 37
snake_y db 12, 12, 12, 12
length  db 4
dir      db 1

start:
    mov ax, 0013h
    int 10h

main_loop:
    call input
    call update
    call draw
    call delay
    jmp main_loop

; --- SAISIE CLAVIER ---
input:
    mov ah, 01h
    int 16h
    jz no_key

    mov ah, 00h
    int 16h

    cmp al, 1Bh
    je exit_game

    cmp ah, 4Dh
    je set_right
```

```

    cmp ah, 4Bh
    je set_left
    cmp ah, 48h
    je set_up
    cmp ah, 50h
    je set_down
    ret

exit_game:
    mov ax, 4C00h
    int 21h

set_right:
    cmp byte [dir], 2
    je no_key
    mov byte [dir], 1
    ret

set_left:
    cmp byte [dir], 1
    je no_key
    mov byte [dir], 2
    ret

set_up:
    cmp byte [dir], 4
    je no_key
    mov byte [dir], 3
    ret

set_down:
    cmp byte [dir], 3
    je no_key
    mov byte [dir], 4
    ret

no_key:
    ret

; --- MISE À JOUR POSITION ---
update:

```

```

    mov cl, byte [length]
    xor ch, ch
    dec cx
    jz update_head

    mov si, cx
shift_loop:
    mov di, si
    dec di
    mov al, [snake_x + di]
    mov [snake_x + si], al
    mov al, [snake_y + di]
    mov [snake_y + si], al
    dec si
    jnz shift_loop

update_head:
    mov al, [dir]
    cmp al, 1
    je move_r
    cmp al, 2
    je move_l
    cmp al, 3
    je move_u
    cmp al, 4
    je move_d
    ret

move_r:
    inc byte [snake_x]
    ret
move_l:
    dec byte [snake_x]
    ret
move_u:
    dec byte [snake_y]
    ret
move_d:

```

```

    inc byte [snake_y]
    ret

; --- AFFICHAGE ---
draw:
    mov ax, 0A000h
    mov es, ax
    xor di, di
    mov cx, 64000
    xor al, al
    rep stosb

    mov cl, byte [length]
    xor ch, ch
    xor si, si

draw_loop:
    ; Calcul offset = y * 320 + x
    mov al, [snake_y + si]
    xor ah, ah
    mov bx, 320
    mul bx
    mov bx, ax

    mov al, [snake_x + si]
    xor ah, ah
    add bx, ax

    mov di, bx
    mov byte [es:di], 10 ; Couleur verte

    inc si
    dec cx
    jnz draw_loop
    ret

; --- TEMPORISATION ---
delay:

```

```
    push cx
    mov cx, 0FFFFh
d1:
    loop d1
    pop cx
    ret
```

4. Compilation et exécution

Dans le terminal (NASM) :

```
cd C:\dos\snake
nasm -f bin snake.asm -o snake.com
```

Dans DOSBox :

```
snake.com
```

5. Résultat attendu

Un serpent s'affiche à l'écran et peut être contrôlé avec le clavier.

6. Travail demandé

L'étudiant doit corriger et améliorer le programme en ajoutant :

- Système de nourriture
- Détection de collision
- Score
- Écran de fin (Game Over)

7. Remise du projet

L'étudiant doit rendre sur classroom:

- Code complet [snake_full.asm](#)
- Vidéo courte de démo commentée